Syrve Integration

1. Overview

1.1 Description

Syrve Integration connects **Syrve** (POS/CRM platform for restaurants and delivery) with **Newo Platform**.

It enables:

- Creating food delivery orders programmatically through conversational AI
- Checking existing order and delivery status in real-time
- Caching and synchronizing restaurant menus (products, sizes, modifiers, pricing)
- Looking up customers by phone via Syrve loyalty system
- Multi-location order routing via terminal groups
- Fuzzy product matching from cached menu using AKB (Agent Knowledge Base)

This integration is designed for **restaurants and food delivery businesses** who need **automated order taking and status tracking through a conversational AI agent** (voice or chat).

1.2 Use Cases

- When a customer wants to place a delivery order → AI extracts order details from conversation, matches menu items, creates order in Syrve
- When a customer asks about order status → AI looks up customer by phone, retrieves orders and delivery info
- When a conversation starts → Menu is automatically refreshed if cache is outdated (24h default)
- When a customer mentions a menu item → Fuzzy search matches the item from cached menu with product ID and price

1.3 Supported Features

Feature	Supported	Notes
Create Delivery Order	✅	LLM payload extraction + fuzzy menu matching
Check Order Status	✅	By customer phone → order ID → delivery status
Menu Synchronization	✅	Auto-cache with 24h TTL, products + sizes + modifiers
Customer Lookup	✅	Via Syrve loyalty by phone number
Terminal Group Routing	✅	Auto-selects first available terminal group
Fuzzy Product Matching	✅	AKB-based search, 0.6 threshold
Modifier Support	✅	Menu cached with modifier groups and prices
Order Context Injection	✅	Order info injected into prompt after creation
Table Reservation	❌	Not implemented
Payment Processing	❌	Not implemented
Order Modification	❌	Create-only, no edits after submission

Multi-Item Orders	✅	Supports array of order items

2. Requirements

Before installation:

- Active **Syrve** account with API access
- **Syrve API key** (apiLogin credential)
- At least one organization and terminal group configured in Syrve
- Menu/nomenclature populated in Syrve

3. How to Setup

3.1 Step 1 — Get API Key

1. Log in to your Syrve management console
2. Navigate to API settings and obtain your API key (apiLogin)
3. Note the API region (default: EU — `https://api-eu.syrve.live`)

3.2 Step 2 — Connect in Platform

1. Open **Newo → Projects**
2. Set the following attributes in **Builder / Attributes**:

Attribute	Required	Description
`syrve_api_key`	✅	Syrve API key (apiLogin)
`syrve_base_url`	❌	API base URL (default: `https://api-eu.syrve.live`)
`syrve_delivery_menu_update_interval`	❌	Menu cache TTL in seconds (default: `86400` = 24 hours)
`syrve_override_agent_attributes`	❌	Override SuperAgent attribute schemas (default: `True`)

3. Click **Save + Publish All**
4. On publish, the SetupFlow automatically:
    - Exchanges the API key for an access token
    - Discovers the organization ID
    - Creates HTTP connectors for API and token operations
    - Registers order creation and status check tools with the agent

4. How to Use

This section explains how the integration works, how to configure automation, and how to test it.

4.1 How the Integration Works

The integration uses Syrve's REST API for order management, customer loyalty, and menu data. Orders are created through LLM-powered payload extraction from conversation context, with fuzzy matching against the cached menu. The menu is auto-refreshed on session start when the cache expires.

- A **Trigger** is a system event that starts a flow

- An **Action** is the API operation performed in Syrve

Available Triggers

Trigger	Event	Description
Create Order	`syrve_create_order_tool`	When the agent needs to place a delivery order
Check Order Status	`check_order_status_tool`	When the agent needs to look up order status
Menu Sync	`session_started`	Auto-refresh menu on conversation start
Menu Sync (manual)	`syrve_get_menu_tool`	Manual menu refresh trigger
Setup	`project_publish_finish`	Initializes integration on deploy

Available Actions

- **Get Token** — Exchange API key for access token (POST `/api/1/access_token`)
- **Get Organizations** — Discover org ID (POST `/api/1/organizations`)
- **Get Menu** — Fetch full nomenclature (POST `/api/1/nomenclature`)
- **Get Terminal Groups** — List order routing points (POST `/api/1/terminal_groups`)
- **Customer Lookup** — Find customer by phone (POST `/api/1/loyalty/syrve/customer/info`)
- **Create Order** — Submit delivery order (POST `/api/1/order/create`)
- **Get Orders** — Retrieve orders by ID (POST `/api/1/order/by_id`)
- **Get Delivery Status** — Check delivery info (POST `/api/1/deliveries/by_id`)

4.2 Architecture Diagrams

Overall Sequence

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as Syrve Integration
    participant API as Syrve API

    Note over I,API: Setup (on publish)
    A->>I: project_publish_finish
    I->>API: POST /api/1/access_token (apiLogin)
    API-->>I: access_token
    I->>API: POST /api/1/organizations
    API-->>I: organization_id
    I->>A: Tools registered

    Note over I,API: Menu Sync (on session start)
    A->>I: session_started
    I->>I: Check menu cache (24h TTL)
    alt Cache outdated
        I->>API: POST /api/1/nomenclature (organizationId)
        API-->>I: Products, sizes, modifiers
        I->>I: Parse, filter deleted, store in cache + AKB
    end
```

```
    Note over U,API: Create Order
    U->>A: "I'd like a margherita pizza delivered"
    A->>I: syrve_create_order_tool (memory)
    I->>I: LLM extract payload (name, phone, address, items)
    I->>API: POST /loyalty/syrve/customer/info (phone)
    API-->>I: customer_id
    I->>API: POST /api/1/terminal_groups (organizationIds)
    API-->>I: terminal_group_id
    I->>I: Fuzzy match "margherita pizza" -> product_id + price
    I->>API: POST /api/1/order/create (order payload)
    API-->>I: orderInfo.id
    I->>I: Inject order into prompt context
    I->>A: result_success (order_id)
    A->>U: "Your order has been placed!"

    Note over U,API: Check Order Status
    U->>A: "What's the status of my order?"
    A->>I: check_order_status_tool (memory)
    I->>I: LLM extract customer info
    I->>API: POST /loyalty/syrve/customer/info (phone)
    API-->>I: customer_id
    I->>API: POST /api/1/deliveries/by_id (posOrderIds)
    API-->>I: Delivery status
    I->>A: result_success (order_id, status)
    A->>U: "Your order is being delivered"
```

CreateOrderFlow

```
flowchart TD
    E["syrve_create_order_tool"] --> LLM["LLM extract payload:<br>name, phone,
address,<br>item, amount, comment"]
    LLM --> VALID{{"Payload<br>valid?"}}
    VALID -->|"No"| ERR["Error:<br>empty_payload"]
    VALID -->|"Yes"| CUST["POST /loyalty/syrve<br>/customer/info<br>{phone}"]
    CUST --> FOUND{{"Customer<br>found?"}}
    FOUND -->|"No"| ERR2["Error:<br>no_customer_found"]
    FOUND -->|"Yes"| TERM["POST /api/1<br>/terminal_groups<br>{organizationIds}"]
    TERM --> MENU["Fuzzy search AKB:<br>syrve_menu label<br>(threshold: 0.6)"]
    MENU --> MATCH{{"Product<br>found?"}}
    MATCH -->|"No"| ERR3["Error:<br>product_not_found"]
    MATCH -->|"Yes"| BUILD["Build order items:<br>productId, amount,<br>price,
comment"]
    BUILD --> ORDER["POST /api/1<br>/order/create<br>{org, terminal,<br>customer,
items,<br>sourceKey: voice-agent}"]
    ORDER --> OK{{"Order ID<br>in response?"}}
    OK -->|"No"| ERR4["Error:<br>missing_order_id"]
    OK -->|"Yes"| INJECT["Inject ExistingOrderInfo<br>into prompt context"]
    INJECT --> OUT["result_success<br>{order_id}"]
```

CollectMenuFlow

```
flowchart TD
    E["session_started<br>or syrve_get_menu_tool"] --> TOKEN{{"Access
token<br>exists?"}}
    TOKEN -->|"No"| SKIP["Skip"]
    TOKEN -->|"Yes"| CACHE{{"Menu cache<br>outdated?<br>(24h TTL)"}}
    CACHE -->|"No"| SKIP
    CACHE -->|"Yes"| API["POST /api/1<br>/nomenclature<br>{organizationId}"]
    API --> PARSE["Parse response:<br>filter deleted items<br>extract sizes +
prices<br>extract modifier groups"]
    PARSE --> STORE["Store menu JSON<br>in syrve_delivery_menu<br>Update timestamp"]
    STORE --> AKB["Update AKB with<br>product summaries<br>(label: syrve_menu)"]
    AKB --> DONE["Menu ready"]
```

CheckOrdersFlow

```
flowchart TD
    E["check_order_status_tool"] --> LLM["LLM extract:<br>name, phone, address"]
    LLM --> CUST["POST /loyalty/syrve<br>/customer/info<br>{phone}"]
    CUST --> FOUND{{"Customer<br>found?"}}
    FOUND -->|"No"| ERR["Error:<br>no_customer_found"]
    FOUND -->|"Yes"| ORDERS["POST /api/1<br>/order/by_id<br>{organizationId}"]
    ORDERS --> HAS{{"Orders<br>exist?"}}
    HAS -->|"No"| ERR2["Error:<br>no_orders_found"]
    HAS -->|"Yes"| STATUS["POST /api/1<br>/deliveries/by_id<br>{posOrderIds}"]
    STATUS --> OUT["result_success<br>{order_id, status,<br>external_number}"]
```

UtilsFlow (Token Refresh)

```
flowchart TD
    REQ["Any API request"] --> HTTP["Send via<br>syrve_connector"]
    HTTP --> STATUS{{"HTTP<br>status?"}}
    STATUS -->|"200"| CHECK{{"Response has<br>error?"}}
    CHECK -->|"No"| SUCCESS["syrve_result_success"]
    CHECK -->|"Yes"| ERR["syrve_result_error"]
    STATUS -->|"401"| REFRESH["POST /api/1<br>/access_token<br>{apiLogin}"]
    REFRESH --> TOK_OK{{"New token?"}}
    TOK_OK -->|"Yes"| SAVE["Save token<br>Retry request"]
    TOK_OK -->|"No"| ERR
    STATUS -->|"4xx/5xx"| ERR
```

4.3 Where to Test

Testing can be done through the **Newo conversation interface** (voice or chat). Each flow (CreateOrder, CheckOrders, CollectMenu) has a dedicated webhook test endpoint for connectivity validation.

4.4 Testing and Verification

To test the integration:

1. **Setup**: Publish the project and verify token exchange + organization discovery succeeds

2. **Menu**: Start a conversation — verify menu is synced (check `syrve_delivery_menu` attribute is populated)
3. **Create Order**: Say "I want to order a [menu item] to [address]" — verify order created in Syrve with correct items and pricing
4. **Check Status**: Ask "What's the status of my order?" — verify delivery status returned

If no action occurs:

- Ensure `syrve_api_key` is set correctly
- Verify `syrve_access_token` was obtained (check it's not empty after setup)
- Confirm `syrve_organization_id` was discovered
- Check that menu cache is populated (`syrve_delivery_menu` is not `[]`)
- Verify the menu was synced to AKB with label `syrve_menu`
- Re-publish the project if credentials were changed

> *Note: This integration creates real orders in Syrve. Orders submitted through the agent are sent to the live POS system with `sourceKey: "voice-agent"`.*

5. FAQ

Q: How does the agent match menu items from conversation? A: The integration uses a two-step process: (1) LLM extracts the item name from conversation memory, (2) fuzzy AKB search with `syrve_menu` label matches the name to cached products (threshold: 0.6). The matched product's ID and price are used for the order.

Q: How often is the menu refreshed? A: By default every 24 hours (`syrve_delivery_menu_update_interval = 86400` seconds). The cache is checked on every `session_started` event. If the timestamp exceeds the TTL, the menu is re-fetched from Syrve's nomenclature API.

Q: What happens if a menu item isn't found? A: The agent receives a `product_not_found` error, which it can communicate to the customer. Only items present in the Syrve nomenclature and cached in AKB can be matched.

Q: Does the integration support modifiers (e.g., extra cheese)? A: The menu cache includes full modifier group data (group names, required/optional flags, min/max counts, individual modifiers with prices). However, the current order creation flow sends an empty `modifiers: []` array — modifier selection is not yet implemented in the ordering pipeline.

Q: Which LLM models are used? A: The primary model is `gemini25_flash` (Google Gemini 2.5 Flash) for most operations. The CheckOrdersFlow's `_getContactSuccessSkill` uses `gpt4` (OpenAI) — this is the only skill in the integration that uses a different model.

Q: How does multi-location routing work? A: During order creation, the integration fetches terminal groups for the organization and automatically selects the first available terminal group. The `syrve_terminal_group_id` is stored and used for order routing.

Q: What customer information is required for ordering? A: First name, phone number (E.164 format), and delivery address are required. Last name is optional — if missing, the first name is copied to the last name field. The phone is used to look up the customer in Syrve's loyalty system.

Q: What is injected into the prompt after order creation? A: The full `orderInfo` object from the Syrve API response is injected as a custom prompt section (`ExistingOrderInfo role="context"`), allowing the agent to reference order details (ID, status, items) in subsequent conversation turns.